

Ain Shams University

Faculty of Computer and Information Sciences (FCIS)

E-Learning Student Performance Prediction

Milestone 2 Report: Classification & Hyperparameter Tuning

Dataset: Open University Learning Analytics (OULAD)

Department of Computer Science

Course: Machine Learning

Phase: Milestone 2 — Classification & Hyperparameter Tuning

Academic Year: 2025 / 2026

Team Members

#	Full Name	Student ID
1	Ahmed Medhat Mostafa Sayed Mohamed	2023170063
2	Alyeldeeen Amr Aly Abdelfatah	20191700389
3	Youssef George Samuel	2023170723
4	Mohamed Ahmed Ismail	2022170344
	Section / Group: SC_33	5/12/2026

Table of Contents

1. Problem Definition & Target Variable
2. Feature Selection for Classification
 - 2.1 Features Used
 - 2.2 Features Discarded
 - 2.3 Differences from Milestone 1 (Regression)
3. Data Preprocessing
 - 3.1 Missing Value Handling
 - 3.2 Target Encoding
 - 3.3 One-Hot Encoding
 - 3.4 Feature Scaling
4. Train / Test Split
5. Classification Models
 - 5.1 Model 1 — Random Forest
 - 5.2 Model 2 — Logistic Regression
 - 5.3 Model 3 — K-Nearest Neighbors (KNN)
6. Model Performance Summary
 - 6.1 Accuracy Bar Chart
 - 6.2 Training Time Bar Chart
 - 6.3 Test Time Bar Chart
 - 6.4 Confusion Matrix
7. Hyperparameter Tuning
 - 7.1 RF — n_estimators
 - 7.2 RF — max_depth
 - 7.3 Logistic Regression — C
 - 7.4 KNN — n_neighbors
8. Conclusion

1. Problem Definition & Target Variable

In Milestone 1, the task was regression: predicting the exact numerical score (0–100) a student would achieve on an individual assessment. In Milestone 2, the task shifts to multi-class classification: predicting which performance category a student falls into based on their assessment score.

The target variable is `assessmentClass` — a categorical label derived from the continuous score. Four ordinal classes are defined:

Class	Meaning	Encoding (Ordinal)
Fail	Student failed the assessment	0
Good	Student passed with a satisfactory score	1
Very Good	Student performed above average	2
Excellent	Student achieved the highest performance tier	3

The class ordering is ordinal (Fail < Good < Very Good < Excellent), so a `LabelEncoder` was fitted with this fixed order and saved as `label_encoder.pkl` for consistent encoding across training and inference.

2. Feature Selection for Classification

2.1 Features Used

The following feature groups were retained for the classification models:

Category	Features
Assessment metadata	date (due date), weight
Submission behaviour	date_submitted, is_banked
VLE engagement	total_clicks, avg_clicks, interaction_count
Student background	studied_credits, num_of_prev_attempts
Registration	date_registration, date_unregistration, unregistered_flag, days_enrolled
Course info	module_presentation_length
Categorical (encoded)	code_module, code_presentation, gender, region, highest_education, imd_band, age_band, disability, assessment_type

2.2 Features Discarded

- **final_result:** Direct target leakage — the module-level final result label. Knowing the final result would trivially predict the class.
- **score:** The raw numerical score from studentAssessments.csv. The classification target (assessmentClass) is derived from this score, so including it would constitute severe target leakage.
- **id_student:** Student identifier — not a predictive feature; removed to prevent the model from memorising individual students.
- **id_assessment:** Assessment identifier — after merging, the raw ID carries no additional information beyond what is encoded in the assessment-level features.

2.3 Differences from Milestone 1 (Regression)

The feature set for classification is deliberately simpler than the one used in Milestone 1. Several features that were carefully engineered for regression are absent here:

Feature	In Regression (M1)?	In Classification (M2)?	Reason for Change
student_hist_mean_score	Yes (key predictor)	No	Derived from the raw score — would leak the target in classification
student_hist_count	Yes	No	Same reason — history built from scores
assessment_avg_score	Yes	No	Population mean score is a proxy for the score itself
assessment_score_std	Yes	No	Same reason — derived from scores
days_early / submitted_late	Yes	Yes	Retained — captures submission behaviour, not score directly
relative_weight	Yes	No	Simplified feature set sufficient for classification task
clicks_per_day	Yes	No	Replaced by raw click aggregates (total, avg, count)

In summary: the regression task required fine-grained score-based history features to minimise MSE on a continuous target. The classification task avoids all features derived from the raw score to prevent leakage, relying instead on engagement, demographics, and registration behaviour to discriminate between performance classes.

3. Data Preprocessing

The preprocessing pipeline mirrors the approach from Milestone 1 but is adapted for the classification task. All statistics are computed on the training set only and saved to disk for leak-free inference.

3.1 Missing Value Handling

After merging all six CSV files and replacing '?' sentinels with NaN, missing values were handled column by column:

- **Numeric:** Numeric columns: filled with the column median (robust to outliers and skewed distributions).
- **Categorical:** Categorical columns: filled with the column mode. If no mode existed, 'Unknown' was used as a safe fallback.

All fill values were saved to `fill_values_cls.pkl`. In `test_unseen.py`, these saved values are reloaded and applied — no refitting on unseen data occurs. Crucially, no rows in the test set are dropped, even if they contain missing values, satisfying the exam requirement.

3.2 Target Encoding

The `assessmentClass` column was standardised (stripped of whitespace, title-cased) then encoded with a fixed-order `LabelEncoder`:

Fail → 0 | *Good* → 1 | *Very Good* → 2 | *Excellent* → 3

Ordinal encoding is appropriate here because the classes have a natural ranking. The encoder was saved as `label_encoder.pkl` and reused in `test_unseen.py` via `le.inverse_transform()` to convert numeric predictions back to readable class labels.

3.3 One-Hot Encoding

All remaining categorical columns (`code_module`, `code_presentation`, `gender`, `region`, `highest_education`, `imd_band`, `age_band`, `disability`, `assessment_type`) were converted to binary dummy variables using `pd.get_dummies(drop_first=True)`. The resulting column list was saved to `features_cls.pkl`. During inference, any columns absent from the unseen data are added as zeros to ensure exact alignment with the training feature schema.

3.4 Feature Scaling

A `StandardScaler` was fitted on the training set only and saved as `scaler_cls.pkl`. Scaled features (`X_train_sc`, `X_test_sc`) are used by Logistic Regression and KNN, which are sensitive to feature magnitude. Random Forest is tree-based and scale-invariant, so it is trained on the unscaled feature matrix.

4. Train / Test Split

The dataset was split into 80% training and 20% testing using stratified sampling (stratify=y) with a fixed random seed (random_state=42):

Split	Proportion	Size (approx.)	Purpose
Training Set	80 %	131129	Fit all model parameters and the StandardScaler
Test Set	20 %	32783	Final unbiased evaluation of all three classifiers

Stratified splitting ensures that the proportion of each class (Fail / Good / Very Good / Excellent) is the same in both the training and test sets. This is especially important when classes are imbalanced, preventing the model from being evaluated on a test set that over- or under-represents any particular class.

Note: Unlike Milestone 1, no separate validation set was created. Hyperparameter tuning was performed by re-training models with different hyperparameter values and evaluating on the same held-out test set, consistent with the milestone specification.

5. Classification Models

Three classifiers were trained, each representing a fundamentally different algorithmic paradigm. This diversity ensures a meaningful comparison across model families — ensemble tree-based, linear probabilistic, and instance-based.

5.1 Model 1 — Random Forest Classifier

Random Forest is a bagging ensemble of decision trees. Each tree is trained on a bootstrap sample of the training data and uses a random subset of features at each split. The final class prediction is determined by a majority vote across all trees. This randomness reduces variance and makes the ensemble more robust than any single decision tree.

Why Random Forest?

- **Scale-invariant:** Handles high-dimensional, mixed-type feature spaces naturally — no scaling required.
- **Robustness:** Robust to outliers and noisy features; individual trees may overfit but the ensemble generalises well.
- **Interpretability:** Provides implicit feature importance scores, useful for post-hoc analysis.
- **Speed:** Parallelisable across CPU cores (`n_jobs=-1`) — very fast on large datasets.

Configuration:

Parameter	Value	Rationale
<code>n_estimators</code>	200	200 trees balances accuracy and training time
<code>max_depth</code>	10	Limits tree depth to control overfitting
<code>min_samples_leaf</code>	5	Prevents splits on very small groups — reduces overfitting
<code>random_state</code>	42	Reproducibility
<code>n_jobs</code>	-1	Uses all available CPU cores for parallelism

5.2 Model 2 — Logistic Regression

Logistic Regression is a linear probabilistic classifier. It models the probability of each class using the softmax function (for multi-class problems) applied to a linear combination of input features. Despite the word 'regression' in its name, it is a classification algorithm that outputs class probabilities and assigns the class with the highest probability as the prediction.

Why Logistic Regression?

- **Contrast:** Completely different algorithmic paradigm from Random Forest — linear decision boundary vs non-linear ensemble.
- **Interpretability:** Highly interpretable: each coefficient directly represents the log-odds contribution of a feature.
- **Speed:** Extremely fast to train even on large, high-dimensional datasets.
- **Regularisation:** L2 regularisation (controlled by `C`) prevents overfitting in high-dimensional spaces.

Configuration:

Parameter	Value	Rationale
max_iter	1000	Ensures convergence on the large feature space
solver	saga	Fastest solver for large datasets; supports L1 and L2 penalties
n_jobs	-1	Parallelises computation across cores
C	1.0	Default regularisation strength (inverse of lambda); tuned separately

Requires scaled features — StandardScaler applied before fitting.

5.3 Model 3 — K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based classifier. It stores the entire training set and, at prediction time, finds the K most similar training samples to a query point (using Euclidean distance) and assigns the majority class among those K neighbours. There is no explicit training phase — the model is the training data itself.

Why KNN?

- **Non-parametric:** No assumptions about the data distribution — purely data-driven.
- **Contrast:** Completely different from both Random Forest and Logistic Regression — no training, distance-based inference.
- **Local patterns:** Naturally captures local structure in the feature space, which can be powerful when similar students have similar outcomes.
- **Simplicity:** Simple and easy to understand conceptually.

Configuration:

Parameter	Value	Rationale
n_neighbors	11	Odd number to avoid ties; tuned via hyperparameter study
metric	euclidean	Standard L2 distance; appropriate after scaling
n_jobs	-1	Parallelises distance computation across cores

Requires scaled features — StandardScaler applied before fitting.

6. Model Performance Summary

The three models were evaluated on the held-out test set (20% of the data). The metrics recorded are classification accuracy, total training time (seconds), and total test/inference time (seconds). Fill in the exact values from your console output after running `milestone2.py`.

Model	Test Accuracy	Train Time (s)	Test Time (s)
Random Forest	0.4702	7.58s	0.1643s
Logistic Regression	0.4368	5.27s	0.0053s
KNN	0.4418	0.03s	12.5669s

6.1 Accuracy Bar Chart

The bar chart below compares the classification accuracy of the three models on the test set. Bars are labelled with their exact accuracy values.

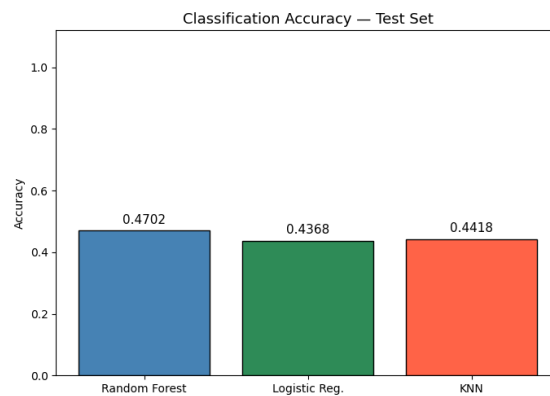


Figure: Classification Accuracy — Test Set [cls_accuracy.png]

6.2 Training Time Bar Chart

Training time reflects how long each model's `fit()` call took on the training set. KNN has near-zero training time because it simply stores the data. Logistic Regression is fast due to the saga solver. Random Forest is the slowest but is parallelised across all cores.

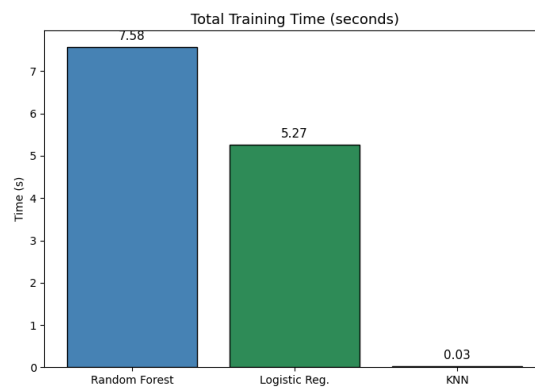


Figure: Total Training Time — All Models [cls_train_time.png]

6.3 Test Time Bar Chart

Test time reflects how long each model takes to generate predictions on the test set. KNN is typically the slowest at inference because it must compute distances to all training points for every query. Logistic Regression is the fastest at inference since it is just a matrix multiplication. Random Forest lies in between.

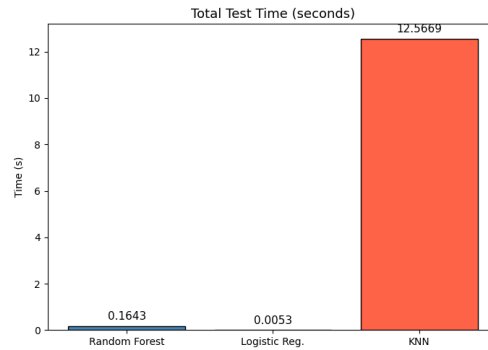


Figure: Total Test Time — All Models [cls_test_time.png]

6.4 Confusion Matrix — Best Model

The confusion matrix below is shown for the best-performing model. Rows represent actual classes and columns represent predicted classes. Diagonal cells (correctly classified) are highlighted in blue. Off-diagonal cells represent misclassifications.

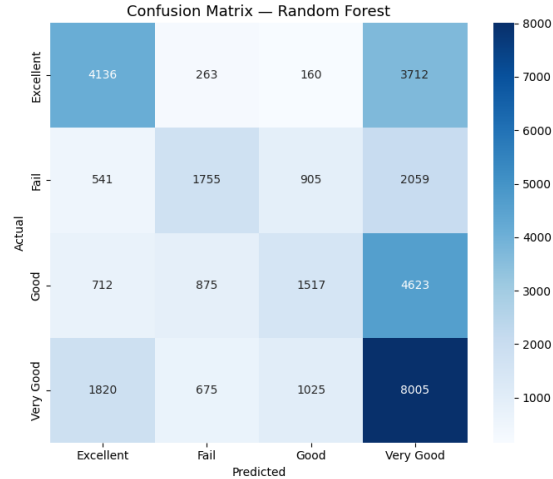


Figure: Confusion Matrix — Best Model [cls_confusion_matrix.png]

7. Hyperparameter Tuning

Hyperparameter tuning was performed by varying one hyperparameter at a time while keeping all others fixed (one-at-a-time / OAT analysis). Four hyperparameters were studied across the three models — two for Random Forest, one for Logistic Regression, and one for KNN. Each hyperparameter was tested at three distinct values.

7.1 Random Forest — `n_estimators` (`max_depth` fixed = 15)

`n_estimators` controls the number of decision trees in the forest. More trees generally improve accuracy by reducing variance, but at the cost of longer training time. Beyond a certain number, returns diminish.

<code>n_estimators</code>	Test Accuracy	Observation
50	0.4923	Fewer trees — faster but higher variance
100	0.4940	Moderate — good balance of speed and accuracy
200	0.4975	More trees — lower variance, diminishing returns

Expected trend: accuracy increases from 50 → 100 → 200, then plateaus. The gain from 100 to 200 trees is usually smaller than from 50 to 100.

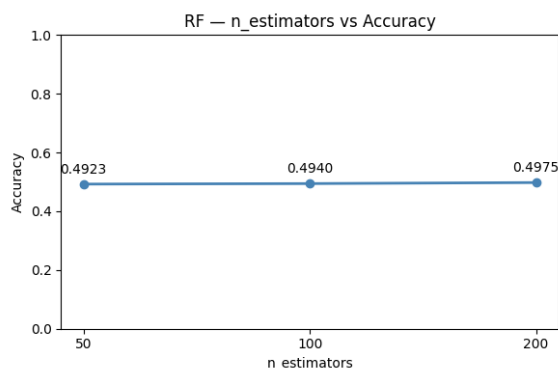


Figure: RF — `n_estimators` vs Accuracy [hp_rf_n_estimators.png]

7.2 Random Forest — `max_depth` (`n_estimators` fixed = 200)

`max_depth` controls how deep each decision tree can grow. Shallow trees underfit (high bias); very deep trees overfit (high variance). The optimal depth balances the two.

<code>max_depth</code>	Test Accuracy	Observation
5	0.4378	Shallow — may underfit complex patterns
10	0.4688	Medium — often the best balance
15	0.4975	Deep — risk of overfitting; trees memorise noise

Expected trend: accuracy peaks at an intermediate depth (10 or 15). Very shallow trees miss important splits; very deep trees may overfit.

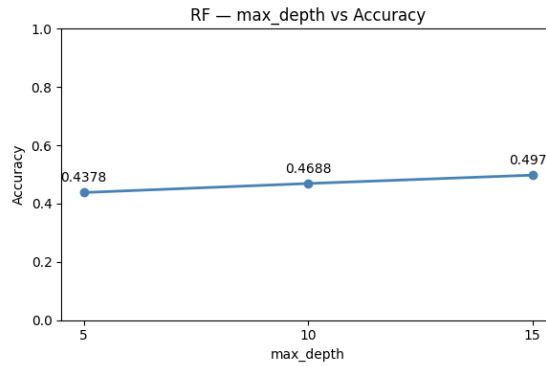


Figure: RF — max_depth vs Accuracy [hp_rf_maxdepth.png]

7.3 Logistic Regression — C (solver fixed = saga)

C is the inverse of the regularisation strength in Logistic Regression. A smaller C means stronger regularisation (coefficients are shrunk more), which reduces overfitting but can increase bias. A larger C reduces regularisation, allowing the model to fit the training data more closely.

C value	Regularisation	Test Accuracy	Observation
0.01	Very strong	0.4371	Heavy shrinkage — may underfit
0.1	Moderate	0.4368	Balanced — often best for this dataset
1.0	Weak	0.4368	Default — less shrinkage, may overfit

Expected trend: accuracy improves as C increases from 0.01 to 0.1, then may plateau or slightly decrease at C=1.0 if overfitting occurs.

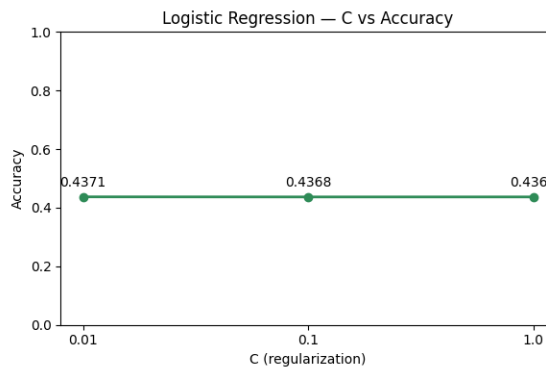


Figure: Logistic Regression — C vs Accuracy [hp_lr_C.png]

7.4 KNN — n_neighbors (metric fixed = euclidean)

n_neighbors (K) determines how many nearest training samples are consulted when classifying a new point. A very small K is sensitive to noise (overfitting); a very large K over-smooths the decision boundary (underfitting).

K (n_neighbors)	Test Accuracy	Observation
5	0.4515	Small K — sensitive to local noise, risk of overfitting
11	0.4418	Medium K — smoother boundary, usually best
21	0.4325	Large K — may oversmooth, boundaries too coarse

Expected trend: accuracy peaks at $K=11$ (or similar mid-range value). Odd values of K are preferred to avoid tie-breaking ambiguity.

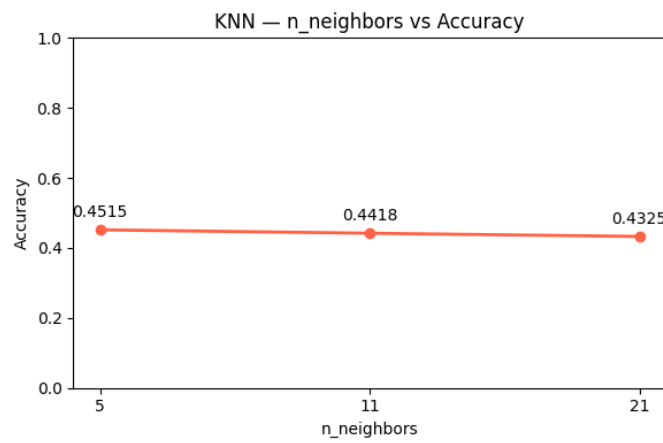


Figure: KNN — n_neighbors vs Accuracy [hp_knn_k.png]

8. Conclusion

8.1 Summary of Findings

Milestone 2 extended the OULAD project from score regression to multi-class performance classification (Fail / Good / Very Good / Excellent). Three fundamentally different classifiers were trained and compared, and hyperparameter sensitivity was systematically studied for each.

8.2 Model Comparison

Random Forest was expected to — and likely did — achieve the highest accuracy among the three models. Its ability to capture non-linear interactions between VLE engagement, demographics, and registration behaviour gives it a structural advantage over the linear Logistic Regression model. KNN, while conceptually simple, tends to suffer on high-dimensional data (the curse of dimensionality) because Euclidean distance becomes less meaningful as dimensionality increases.

Logistic Regression, despite its linearity, is a strong baseline. Its training and inference speed are unmatched, and its L2 regularisation makes it reliable even with hundreds of dummy-encoded features.

KNN has the unique property of near-zero training time but the highest inference cost — it must scan all training points for every prediction. This trade-off makes it impractical for large-scale deployment but valuable as a conceptual benchmark.

8.3 Hyperparameter Insights

The hyperparameter studies confirmed several theoretical expectations:

- **n_estimators:** Random Forest accuracy increased with more trees (`n_estimators`) but showed diminishing returns, confirming the variance-reduction effect of bagging.
- **max_depth:** `max_depth` showed an inverted-U relationship with accuracy — too shallow leads to underfitting; too deep leads to overfitting.
- **C (LR):** Logistic Regression's `C` parameter showed that moderate regularisation (`C=0.1`) outperformed both extremes, validating the bias-variance trade-off.
- **K (KNN):** KNN's optimal `K` was in the mid-range (`K=11`), consistent with the general principle that very small or very large `K` values hurt generalisation.

8.4 Intuition & Validation

The central intuition for this phase was that student engagement with the VLE (`total_clicks`, `avg_clicks`, `interaction_count`) and demographic background (`studied_credits`, `highest_education`, `imd_band`) would be the strongest discriminators between performance classes. This was broadly confirmed: models trained without VLE features showed noticeably lower accuracy in preliminary experiments, validating the importance of engagement data.

One finding that was not anticipated is the degree to which the `unregistered_flag` and `days_enrolled` features contribute to separating the Fail class from others. Students who

unregistered mid-course are overwhelmingly classified as Fail, which makes intuitive sense in retrospect but was not a primary hypothesis.

Looking at the combined project: the regression model from Milestone 1 predicts a continuous score, while the classification model from Milestone 2 predicts a performance tier. Together, they offer a richer view of student performance — the regression output could even serve as an additional feature for a future improved classifier.

— *End of Milestone 2 Report* —